

UNIVERSIDADE DO VALE DO ITAJAÍ UNIVALI

Disciplina de Sistemas Distribuídos

Prof. Ramicés dos Santos Silva

TRABALHO 1

Comunicação de Grupo, Ordem Total de Mensagens e Estado Global

Roteiro para as equipes

Equipes de até 4 alunos

Entrega 1 (código + relatório) via AVA até 09/09/2026

Entrega 2 (slides + seminário) em aula em 16/09/2026

1º semestre de 2026

Sumário

1. Apresentação e objetivos	3
Objetivos de aprendizagem.....	3
2. Regras gerais e organização	3
3. Requisitos funcionais (o que será avaliado)	4
4. Temas (escolha um)	4
4.1 Chat distribuído	4
4.2 Servidor de compartilhamento de arquivos distribuído	4
5. Fundamentos técnicos.....	5
5.1 Relógio de Lamport (escalar).....	5
5.2 Relógio vetorial.....	5
Condição de entrega causal (buffer de delivery).....	5
5.3 O ponto central: ordem causal × ordem total	6
Abordagem A Relógio vetorial + desempate determinístico	6
Abordagem B Sequenciador / super-servidor eleito	6
5.4 Eleição de líder (obrigatório se usar a Abordagem B; opcional na A)	6
Algoritmo do Valentão (Bully).....	6
Algoritmo do Anel.....	7
5.5 Estado global.....	7
6. Comunicação em rede sugestões de implementação	8
6.1 Unicast com TCP (ponto a ponto confiável, FIFO)	8
6.2 Multicast com UDP (comunicação de grupo nativa)	8
6.3 Endereçamento o que registrar no relatório	9
7. Ambiente de execução com múltiplos nós	9
8. Tela de cada nó (requisito R5)	10
9. Passo a passo sugerido de implementação	11
10. Relatório estrutura obrigatória	11
11. Entregas e cronograma	11
Cronograma sugerido	12
12. Checklist de autoavaliação antes de entregar	12
13. Dicas e armadilhas comuns	12
14. Referências de apoio	12

1. Apresentação e objetivos

Este trabalho parte do problema clássico de **entrega ordenada de mensagens com relógios vetoriais** e o generaliza para um sistema distribuído de comunicação de grupo com múltiplos nós. Vocês construirão um sistema configurável (possibilite até **15 nós ou mais**) que implementa comunicação de grupo, **ordem total** das mensagens e um mecanismo de **estado global**.

Ao final, cada equipe entrega o código funcional, um relatório técnico detalhado (Entrega 1) e apresenta um seminário (Entrega 2).

Objetivos de aprendizagem

- Compreender e implementar relógios lógicos (Lamport) e relógios vetoriais na prática.
- **Distinguir ordem causal (parcial) de ordem total** e implementar um mecanismo que garanta a ordem total das mensagens difundidas.
- Implementar comunicação de grupo usando primitivas de rede (multicast ou unicast).
- Modelar e capturar um estado global consistente de um sistema distribuído.
- Exercitar (**OPCIONAL**) eleição de líder e o papel de um nó coordenador/sequenciador.
- Projetar, documentar e defender oralmente uma solução distribuída.

2. Regras gerais e organização

- **Equipes:** no máximo 4 alunos por equipe.
- **Comunicação de grupo:** a equipe escolhe entre **multicast** ou **unicast** (a decisão deve ser justificada no relatório).
- **Tema:** a equipe escolhe um entre dois temas: **chat distribuído** ou **servidor de compartilhamento de arquivos distribuído**.
- **Número de nós configurável:** a solução deve permitir escolher quantos nós serão inicializados, suportando **pelo menos 15 nós**.
- **Somente troca de mensagens de rede:** os nós só podem se comunicar por mensagens de rede. **É proibido** usar qualquer memória compartilhada, banco de dados comum, arquivo compartilhado em disco, variável global entre threads ou serviço externo compartilhado como canal de coordenação.

Consequência prática da regra de "somente rede"

Cada nó deve ser um processo do sistema operacional independente, com seu próprio estado em memória privada. Rodar todos os nós como threads dentro de um único processo, compartilhando estruturas de dados, descaracteriza o trabalho.

Exceção permitida: um arquivo/argumento de **configuração estático** (lista de nós com endereço e porta), lido na inicialização, é aceitável ele não é um canal de coordenação em tempo de execução, apenas o "catálogo de endereços" da rede.

3. Requisitos funcionais (o que será avaliado)

A tabela abaixo mapeia cada orientação da atividade em um requisito verificável.

#	Requisito	Como demonstrar
R1	Comunicação de grupo (multicast OU unicast)	Uma mensagem de grupo enviada por um nó chega a todos os demais.
R2	Escolha de tema (chat OU file sharing)	Funcionalidade do tema operante entre os nós.
R3	Nº de nós configurável, ≥ 15	Iniciar a simulação com 3, 8 e 15, nós pelo menos, sem alterar código.
R4	Ordem total via relógio vetorial (Lamport + melhorias)	Todos os nós entregam as mensagens de grupo na MESMA ordem.
R5	Tela do nó	Mostrar: envio p/ nó específico, envio p/ grupo, ordem local e ordem global.
R6	Estado global	Comando que dispara a captura de um estado global consistente e o exibe.
R7	Relatório detalhado	Documento com todos os itens da Seção 10.
R8	Eleição de líder (se adotada)	Passo a passo do algoritmo e demonstração de reeleição ao cair o líder.

4. Temas (escolha um)

4.1 Chat distribuído

Um sistema de bate-papo em que cada nó é um participante. Deve permitir:

- Mensagem privada para um nó específico, identificado por seu ID.
- Mensagem de grupo para todos os participantes.
- **Entrega ordenada:** as mensagens de grupo devem aparecer na mesma ordem em todos os participantes (ordem total).
- Exibir, em cada nó, a ordem local de emissão e a ordem global de entrega.

4.2 Servidor de compartilhamento de arquivos distribuído

Um conjunto de nós que mantém uma visão coerente de um repositório de arquivos replicado/particionado. Deve permitir:

- Operações difundidas ao grupo: publicar/atualizar/remover um arquivo (metadados no mínimo; conteúdo é um diferencial).
- Requisição unicast de um arquivo a um nó específico.
- **Ordem total das operações:** todas as réplicas aplicam as operações na mesma sequência, evitando divergência entre réplicas.
- Exibir a ordem local e a ordem global das operações e o estado do índice de arquivos em cada nó.

Dica de escopo

Para ambos os temas, o foco da avaliação é o middleware de comunicação e ordenação, não a riqueza da aplicação. Uma interface de terminal (linha de comando) é suficiente. Priorizem a correteza da ordem total e do estado global antes de investir em recursos extras.

5. Fundamentos técnicos

5.1 Relógio de Lamport (escalar)

Cada nó mantém um contador inteiro C . Regras:

- **Evento local ou envio:** $C = C + 1$; o valor de C acompanha a mensagem.
- **Recepção de mensagem com carimbo C_m :** $C = \max(C, C_m) + 1$.

Limitação: o relógio escalar dá uma ordenação consistente com a causalidade, mas **não permite detectar** se dois eventos são concorrentes ou causalmente relacionados. Daí a necessidade do relógio vetorial.

5.2 Relógio vetorial

Cada nó i mantém um vetor V de tamanho N (número de nós). $V[j]$ é o que o nó i sabe sobre a atividade do nó j . Regras (pseudocódigo):

```
# no nó i
ao_enviar():
    V[i] = V[i] + 1          # conta o próprio evento de envio
    anexa cópia de V à mensagem

ao_receber(msg com vetor Vm de origem k):
    # (recepção NÃO incrementa o vetor de imediato;
    # primeiro decide-se se a msg pode ser entregue)
    para cada j: V[j] = max(V[j], Vm[j])
    V[i] = V[i] + 1          # evento local de entrega
```

Observação do enunciado original

No problema original os **eventos de recepção não são considerados no incremento** do vetor, mas são importantes para a atualização dos componentes referentes aos outros nós. Vocês podem seguir essa convenção; o essencial é que a política de incremento seja **consistente e documentada** no relatório.

Condição de entrega causal (buffer de delivery)

Uma mensagem m de origem k com carimbo V_m só pode ser entregue no nó i quando:

- $V_m[k] = V[k] + 1$ (é a próxima mensagem esperada de k respeita FIFO da origem), e
- $V_m[j] \leq V[j]$ para todo $j \neq k$ (todas as mensagens que causalmente precedem m já foram entregues).

Enquanto as condições não são satisfeitas, m fica em um **buffer de espera**. A cada entrega, reavalie o buffer, pois novas mensagens podem se tornar entregáveis.

5.3 O ponto central: ordem causal × ordem total

Atenção esta é a decisão de projeto mais importante do trabalho. O relógio vetorial, sozinho, garante **ordem causal**, que é uma ordem **parcial**: mensagens *concorrentes* (sem relação de causa e efeito) podem ser entregues em ordens diferentes em nós diferentes. A atividade, porém, exige **ordem total** todos os nós entregam todas as mensagens de grupo na **mesma sequência**. Portanto é preciso um critério extra além do vetor.

Sua equipe deve escolher UMA das duas abordagens a seguir e justificá-la no relatório.

Abordagem A Relógio vetorial + desempate determinístico

Mantém-se o relógio vetorial para a causalidade e define-se uma função de ordenação total que todos os nós aplicam de forma idêntica. Chave de ordenação sugerida para cada mensagem:

```
chave(m) = ( soma(Vm), id_do_no_de_origem, contador_local_da_origem )
# compara-se por soma dos componentes do vetor (relógio escalar derivado);
# empates são resolvidos pelo id do nó e, então, pelo contador local.
```

- **Como funciona:** cada nó só entrega uma mensagem de grupo quando tem certeza de que nenhuma mensagem com chave menor ainda pode chegar. Isso normalmente exige aguardar ter recebido "algo" (mensagem ou reconhecimento) de todos os nós mecanismo de mensagens estáveis/ACK.
- **Vantagem:** totalmente descentralizado, sem ponto único de falha; fiel ao espírito "via relógio vetorial".
- **Custo:** maior latência de entrega e mais mensagens de controle; exige cuidado para não travar quando um nó fica em silêncio (use mensagens periódicas de "batimento"/ACK).

Abordagem B Sequenciador / super-servidor eleito

Um nó atua como sequenciador: ele recebe as mensagens de grupo, atribui um número de sequência global e as redifunde (ou apenas difunde o número de sequência). Todos entregam na ordem do número de sequência. O sequenciador é escolhido por eleição de líder (Seção 5.5).

- **Como funciona:** o relógio vetorial continua sendo usado/registrado para causalidade e para o relatório, mas quem define a ordem total é o número de sequência do líder.
- **Vantagem:** ordem total simples e eficiente; casa diretamente com o requisito de estado global e eleição de líder.
- **Custo:** o sequenciador é um ponto crítico é obrigatório tratar sua queda com reeleição; distancia-se um pouco do "via relógio vetorial" puro.

O que esperamos no relatório

Independentemente da abordagem, o relatório deve conter o **passo a passo do algoritmo de ordem total escolhido**, com um exemplo numérico mostrando dois nós chegando à mesma ordem global a partir de mensagens concorrentes.

5.4 Eleição de líder (obrigatório se usar a Abordagem B; opcional na A)

Duas opções clássicas descreva no relatório a que usar, com seu passo a passo:

Algoritmo do Valentão (Bully)

1. Ao notar que o líder caiu (timeout), o nó envia ELEIÇÃO a todos os nós com ID maior que o seu.

2. Se ninguém com ID maior responder dentro do timeout, ele se declara líder e envia COORDENADOR a todos.
3. Se algum ID maior responder (OK), ele desiste e aguarda o anúncio do coordenador.
4. O nó de maior ID ativo sempre vence.

Algoritmo do Anel

1. Os nós formam um anel lógico. Quem detecta a falha inicia, circulando uma mensagem de eleição com sua ID.
2. Cada nó adiciona sua ID (ou substitui se a sua for maior) e repassa ao próximo ativo.
3. Quando a mensagem retorna à origem, o maior ID é o novo líder, anunciado em nova volta.

5.5 Estado global

O requisito de estado global pede que o sistema seja capaz de capturar uma "fotografia" coerente de todos os nós. Recomendamos o **algoritmo de snapshot de Chandy-Lamport**, que é a solução canônica vista em aula e respeita a regra de "somente mensagens de rede":

1. Um nó iniciador registra seu próprio estado e envia um MARCADOR por todos os seus canais de saída.
2. Ao receber o primeiro MARCADOR, um nó registra seu estado, marca como vazio o canal por onde o marcador chegou e propaga o MARCADOR pelos seus canais de saída.
3. Para os demais canais, o nó grava as mensagens recebidas entre o registro do seu estado e a chegada do marcador daquele canal (estado do canal).
4. O snapshot termina quando todos os nós registraram estado e todos os canais foram contabilizados.

Justificativa a apresentar

No relatório, **justifique a escolha** do mecanismo de estado global. Chandy-Lamport é indicado por não exigir parar o sistema nem relógio sincronizado, produzindo um estado *consistente*. Se optarem por uma variante centralizada (o líder consulta todos e agrega respostas), justifiquem o trade-off simplicidade contra ponto único de falha dentro dos conceitos de sistemas distribuídos.

6. Comunicação em rede sugestões de implementação

Linguagem sugerida: Python (pela facilidade com sockets e concorrência), mas a escolha é livre Java, Go, C#, Node.js etc. são aceitos, desde que a comunicação seja exclusivamente por rede. Os trechos abaixo são o mínimo para começar; vocês devem envolvê-los em threads, tratamento de erros e serialização de mensagens (sugerimos JSON com campos como tipo, origem, destino, vetor, seq, payload).

6.1 Unicast com TCP (ponto a ponto confiável, FIFO)

TCP é indicado para o unicast porque já oferece entrega confiável e ordem FIFO por canal premissa do problema original.

```
import socket, json

# --- Envio unicast para um nó específico ---
def enviar_unicast(host, porta, mensagem: dict):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((host, porta))
        s.sendall(json.dumps(mensagem).encode('utf-8'))

# --- Recepção unicast: cada nó roda isto em uma thread ---
def servidor_unicast(porta, ao_receber):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(('0.0.0.0', porta))
    s.listen()
    while True:
        conn, _ = s.accept()
        with conn:
            dados = conn.recv(65536)
            if dados:
                ao_receber(json.loads(dados.decode('utf-8')))
```

Para comunicação de grupo via unicast: basta iterar sobre a lista de todos os nós (do arquivo de configuração) e chamar `enviar_unicast` para cada um. Isso dá controle total (e é a base para ACKs individuais na Abordagem A).

6.2 Multicast com UDP (comunicação de grupo nativa)

O multicast IP entrega uma única mensagem a todos os nós inscritos no grupo eficiente para difusão. Use um endereço da faixa multicast (224.0.0.0 a 239.255.255.255).

```
import socket, struct, json

GRUPO = '224.1.1.1'      # endereço multicast do grupo
PORTA = 5007

# --- Emissor: difunde para o grupo ---
def enviar_multicast(mensagem: dict):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
    s.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 2)
    s.sendto(json.dumps(mensagem).encode('utf-8'), (GRUPO, PORTA))
    s.close()

# --- Receptor: cada nó entra no grupo e escuta (rodar em thread) ---
def receptor_multicast(ao_receber):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
```



```
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(('', PORTA))
mreq = struct.pack('4sl', socket.inet_aton(GRUPO), socket.INADDR_ANY)
s.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
while True:
    dados, _ = s.recvfrom(65536)
    ao_receber(json.loads(dados.decode('utf-8')))
```

Cuidado com o UDP

UDP não é confiável: pode perder, duplicar ou reordenar datagramas. Se escolherem multicast, vocês precisam tratar isso na camada de ordenação (números de sequência por origem, detecção de lacunas e retransmissão sob demanda). Documentem essa limitação no relatório.

Ambiente: multicast entre máquinas depende da rede/roteador. Para testes em uma só máquina (todos os nós em localhost), use SO_REUSEADDR/SO_REUSEPORT e TTL baixo. Em VMs/containers, verifiquem se a rede permite multicast.

6.3 Endereçamento o que registrar no relatório

- **Unicast:** cada nó = um IP + porta única. Em uma máquina, use 127.0.0.1 e portas 5000 + id (5001, 5002, ...).
- **Multicast:** um endereço de grupo (ex.: 224.1.1.1:5007) compartilhado por todos os nós; registrem a faixa usada e o TTL.
- Documentem a tabela de endereços/portas dos nós usada nas simulações (parte obrigatória do relatório).

7. Ambiente de execução com múltiplos nós

Como cada nó é um processo independente, você precisa iniciar N processos e informar a cada um sua identidade e a lista de nós. Um **arquivo de configuração** estático resolve isso:

```
// nos.json (catálogo estático de endereços lido na inicialização)
{
  "grupo_multicast": "224.1.1.1",
  "porta_grupo": 5007,
  "nos": [
    {"id": 1, "host": "127.0.0.1", "porta": 5001},
    {"id": 2, "host": "127.0.0.1", "porta": 5002},
    {"id": 3, "host": "127.0.0.1", "porta": 5003}
  ]
}
```

Cada nó é iniciado com seu id, por exemplo: `python no.py --id 3 --config nos.json`. Formas práticas de subir vários nós:

- **Vários terminais / tmux:** um terminal por nó ótimo para ver a tela de cada nó ao vivo (atende ao requisito R5).
- **Script de inicialização:** um script que faz spawn de N processos em background e gera automaticamente o `nos.json` para 3, 8 ou 15 nós.
- **Docker Compose:** um serviço por nó (ou usar réplicas) bom para simular máquinas distintas e testar multicast entre containers.

Sobre concorrência dentro de um nó

Dentro de cada processo-nó, use threads (ou asyncio) para separar: (a) escuta de rede, (b) processamento/entrega ordenada e (c) interface com o usuário. Isso é concorrência *interna* ainda que os nós troquem dados apenas por rede. Proteja as estruturas internas (buffer, relógio vetorial) com locks.

8. Tela de cada nó (requisito R5)

A interface (pode ser terminal) de cada nó deve oferecer e exibir, no mínimo:

- **Enviar para nó específico** (mensagem/operação unicast, escolhendo o ID de destino).
- **Enviar mensagem de grupo** (difusão a todos).
- **Ordem local das mensagens:** a sequência em que este nó emitiu/observou eventos localmente.
- **Ordem global das mensagens:** a fila de delivery já ordenada pela ordem total deve ser idêntica em todos os nós.
- Sugestão extra: exibir o relógio vetorial atual e o buffer de mensagens aguardando entrega.

Critério de corretude

Ao final de uma simulação, **as filas de delivery ordenadas (ordem global) devem ser iguais em todos os nós**. Mostrar essa igualdade como no exemplo dos buffers do enunciado original é a principal evidência de que a ordem total funciona.

9. Passo a passo sugerido de implementação

1. **Definições iniciais:** escolham tema, tipo de comunicação (multicast/unicast) e abordagem de ordem total (A ou B). Criem o repositório e dividam os papéis.
2. **Camada de rede:** implementem envio/recepção (Seção 6) e o formato de mensagem em JSON. Teste: dois nós trocam "olá".
3. **Inicialização de N nós:** leiam o `nos.json`, subam 3 nós, depois validem com 15 (requisito R3).
4. **Relógio vetorial + entrega causal:** implementem o vetor e o buffer de delivery com a condição da Seção 5.2.
5. **Ordem total:** apliquem a Abordagem A (desempate determinístico + estabilidade) ou B (sequenciador). Validem que todos os nós convergem para a mesma ordem.
6. **Eleição de líder:** se usarem a Abordagem B, implementem Bully ou Anel e testem a queda do líder.
7. **Estado global:** implementem o snapshot (Chandy-Lamport) disparável por um comando de menu.
8. **Tela do nó:** completem a interface com ordem local, ordem global e envio unicast/grupo (R5).
9. **Simulações e cenários:** reproduzam um cenário de mensagens concorrentes e demonstrem a ordem total; incluam um caso de dependência causal (ex.: uma mensagem que só é emitida após duas outras, como no enunciado original).
10. **Relatório e testes finais:** documentem tudo (Seção 10) e testem com 15 nós.

10. Relatório estrutura obrigatória

O relatório da Entrega 1 deve conter, no mínimo, as seções abaixo (orientação 8 da atividade):

1. **Descrição do problema** e do tema escolhido.
2. **Papel de cada membro** da equipe na execução do projeto.
3. **Arquitetura da solução** (componentes, diagrama, fluxo de mensagens).
4. **Endereços de rede utilizados** (multicast ou unicast): faixa, IPs, portas e a tabela de nós.
5. **Simulações** realizadas (cenários, nº de nós, prints das filas de delivery mostrando a ordem global idêntica).
6. **Limitações do modelo escolhido** (ex.: perdas em UDP/multicast; ponto único no sequenciador; latência na Abordagem A).
7. **Tecnologia e linguagem** utilizadas e como executar o projeto.
8. **Passo a passo do algoritmo de ordem total** (com exemplo numérico) e, se adotado, do algoritmo de eleição de líder.
9. **Justificativa do estado global** escolhido.

11. Entregas e cronograma

Entrega	O que	Como / Quando
Entrega 1	Código-fonte da solução + relatório detalhado	Via AVA até 09/09/2026
Entrega 2	Slides + apresentação do seminário	Em aula, em 16/09/2026

Cronograma sugerido

Período	Foco	Marco
23–28/ago	Formar equipe; escolher tema, comunicação e abordagem de ordem total; montar repositório e dividir papéis.	Definições fechadas
29/ago–04/set	Camada de rede (multicast/unicast); inicialização de N nós; relógio vetorial + entrega causal.	Protótipo comunicando
05–08/set	Ordem total; estado global; eleição de líder (se Abordagem B); testes com 15 nós; escrever relatório.	Solução completa + relatório
09/set	Revisão final e submissão.	ENTREGA 1 (AVA)
10–15/set	Preparar slides e ensaiar o seminário.	Slides prontos
16/set	Apresentação do seminário.	ENTREGA 2 (aula)

12. Checklist de autoavaliação antes de entregar

- A equipe tem no máximo 4 integrantes e os papéis estão descritos no relatório.
- O sistema inicia com número configurável de nós e foi testado com 15.
- Comunicação de grupo funciona (multicast ou unicast) e está justificada.
- As filas de delivery (ordem global) são idênticas em todos os nós.
- A tela de cada nó mostra envio unicast, envio de grupo, ordem local e ordem global.
- O estado global pode ser capturado e exibido, com a escolha justificada.
- Nenhuma memória/serviço compartilhado é usado só mensagens de rede.
- O relatório cobre todos os itens da Seção 10, com passo a passo dos algoritmos.

13. Dicas e armadilhas comuns

- **Confundir ordem causal com ordem total:** o vetor sozinho não basta releia a Seção 5.3.
- **Entregar cedo demais:** não entregue uma mensagem antes de garantir que nenhuma anterior (na ordem total) ainda pode chegar.
- **Deadlock por silêncio:** na Abordagem A, nós quietos travam a estabilidade; use mensagens periódicas de batimento/ACK.
- **UDP perdendo pacotes:** trate lacunas com números de sequência e retransmissão.
- **Estado compartilhado escondido:** cuidado para não acoplar nós por um arquivo/BD comum isso viola a regra 11.
- **Não tratar a queda do líder:** se usar sequenciador, a reeleição é obrigatória.

14. Referências de apoio

- Coulouris, G. et al. Sistemas Distribuídos: Conceitos e Projeto. (capítulos de tempo/coordenação e multicast confiável/ordenado).
- Tanenbaum, A.; van Steen, M. Sistemas Distribuídos: Princípios e Paradigmas. (relógios lógicos, exclusão mútua, eleição, snapshots).

- Lamport, L. "Time, Clocks, and the Ordering of Events in a Distributed System" (1978).
- Chandy, K. M.; Lamport, L. "Distributed Snapshots: Determining Global States of Distributed Systems" (1985).
- Documentação do módulo socket do Python (multicast e TCP).